

Assignment 9 - **Solution**

Assignment 9 - Objectives:

- x86-64 assembly instructions
- Loop unrolling optimization
- Spatial locality - Can be done after our Lecture 37 on Wed. August 6.
- Microbenchmarking

Requirement for this Assignment 9:

- Always show your work (as illustrated in lectures), if appropriate.

Marking scheme:

- This assignment will be marked for correctness.
- A solution will be posted on Monday after the due date.

Deadline:

- Friday August 8 at 23:59:59 on Crowdmark.
- No extension will be granted unless for medical reasons, once the Official Medical Certificate has been completed and emailed to the instructor before Friday (due day) 5pm.

Late assignments will receive a grade of 0.

Enjoy!

Q1 (7 points)**x86-64 assembly instructions**

Do the following Homework Problems from Chapter 3 of our Textbook:

- 3.65 at page 317
- 3.64 at page 316

3.65 at page 317 - Solution:

It might surprise students that they can understand the machine code implementation of a function based only on its inner loop. It's a useful strategy, since it avoids wading through lots of uninteresting code.

- A. We can see in the code that registers `%rdx` and `%rax` are being used as pointers. The only question is which one represents which matrix element. We can see in line 6 that `%rdx` gets incremented by 8. This must be a pointer to `A[i][j]`.
- B. That leaves `%rax` as a pointer to `A[j][i]`.
- C. Since `%rax` is incremented by 120 on each iteration (line 7), we must have $M = 120/8 = 15$.

3.62 at page 316 - Solution:

- A. Array element $A[i][j][k]$ is located at address $x_A + 8(T(S \cdot i + j) + k)$.
- B. Consider the following annotated version of the assembly code:

```

    long store_ele(long i, long j, long k, long *dest)
    i in %rdi, j in %rsi, k in %rdx, dest in %rcx
1 store_ele:
2  leaq    (%rsi,%rsi,2), %rax    3 j
3  leaq    (%rsi,%rax,4), %rax    13 j
4  movq    %rdi, %rsi            Copy i
5  salq    $6, %rsi             64 i
6  addq    %rsi, %rdi            65 i
7  addq    %rax, %rdi            13 j + 65 i = 13(5 i + j)
8  addq    %rdi, %rdx            13(5 i + j) + k
9  movq    A(,%rdx,8), %rax      M[x_A + 8 (13(5 i + j) + k)]
10 movq    %rax, (%rcx)
11 movl    $3640, %eax
12 ret

```

We can see that memory reference on line 9 indicate that $T = 13$ and $S = 5$. We can see on line 11 that the total array size is 3640 bytes. From this, we get $R = 3640 / (8 \cdot 13 \cdot 5) = 7$.

Q2 (6 points)
Loop unrolling optimization

- a. Below, write a version of the inner product C function described in Homework Problem 5.13 (at page 570 in our textbook) that uses 6 x 1 loop unrolling. Make sure you comment your function. You do not have to answer the questions A., B., C. and D. of this Homework Problem 5.13 in the textbook.

Possible solution:

```

/* 6x1unrolling
 * where k = 6 -> dealing with six (6) elements of arrays u and v
 * at each iteration of the loop
 * where c = 1 -> only one (1) accumulator at each iteration
 * of the loop, i.e., only one (1) running sum (variable in
 * which to "accumulate" the running sum).
 */

void inner6x1unrolling(vec_ptr u, vec_ptr v, data_t *dest) {
    long k = 6;
    long i;
    long length = vec_length(u);
    data_t *udata = get_vec_start(u);
    data_t *vdata = get_vec_start(v);

    // Initialize 1 accumulator
    data_t sum = (data_t) 0;

    // Compute number of iterations considering i += k (6)
    long limit = length - k + 1;

    // Dealing with k (6) elements at a time per iteration
    for (i = 0; i < limit; i += k){
        sum = sum + udata[i]    * vdata[i]
                + udata[i+1] * vdata[i+1]
                + udata[i+2] * vdata[i+2]
                + udata[i+3] * vdata[i+3]
                + udata[i+4] * vdata[i+4]
                + udata[i+5] * vdata[i+5];
    }

    // For cases when length < k OR length not a multiple of k
    for (; i < length; i++) {
        sum = sum + udata[i] * vdata[i];
    }

    *dest = sum;

    return;
}

```

b. Below, write a second version of this inner product function described in Homework Problem 5.13 that uses 6 x 6 loop unrolling. Make sure you comment your function. Again, you do not have to answer the questions A., B., C. and D. of this Homework Problem 5.13 in the textbook.

Possible solution:

```

/* 6x6unrolling
 * where k = 6 -> dealing with six (6) elements of arrays u and v
 * at each iteration of the loop
 * where c = 6 -> using six (6) accumulators at each iteration
 * of the loop, i.e., six (6) running sums (variables in
 * which to "accumulate" the six running sums.
 */

void inner6x6unrolling(vec_ptr u, vec_ptr v, data_t *dest) {
    long k = 6;
    long i;
    long length = vec_length(u);
    data_t *udata = get_vec_start(u);
    data_t *vdata = get_vec_start(v);

    // Initialize all 6 accumulators
    data_t sum0 = (data_t) 0;
    data_t sum1 = (data_t) 0;
    data_t sum2 = (data_t) 0;
    data_t sum3 = (data_t) 0;
    data_t sum4 = (data_t) 0;
    data_t sum5 = (data_t) 0;

    // Compute number of iterations considering i += k (6)
    long limit = length - k + 1;

    // Dealing with k (6) elements at a time per iteration
    for (i = 0; i < limit; i += k) {
        sum0 = sum0 + udata[i] * vdata[i];
        sum1 = sum1 + udata[i+1] * vdata[i+1];
        sum2 = sum2 + udata[i+2] * vdata[i+2];
        sum3 = sum3 + udata[i+3] * vdata[i+3];
        sum4 = sum4 + udata[i+4] * vdata[i+4];
        sum5 = sum5 + udata[i+5] * vdata[i+5];
    }

    // For cases when length < k OR length not a multiple of k
    for (; i < length; i++) {

```

```
    sum0 = sum0 + udata[i] * vdata[i];
}

// Total up all accumulators
*dest = (sum0 + sum1) + (sum2 + sum3) + (sum4 + sum5);
return;
}
```

Q3 (7 points)

Spatial locality - Can be done after our Lecture 37 on Wed. August 6.

Consider the syntactically correct function below:

```
int sumArray(int A[N][N][N]) {
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[k][i][j];

    return sum;
}
```

1. Determine its memory access pattern and stride.

Note: When answering the above question, you may find it helpful to set N to a value greater than 1 to clearly observe the memory access pattern. Then, when writing the stride in your answer, make sure you express the stride in general terms, using N (or an expression of N), rather than using a specific value (such as 3).

Solution

1. Determine the memory access pattern and stride.

```
int sumArray(int A[N][N][N])
{
    int i, j, k, sum = 0;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[k][i][j];
    return sum;
}
```

Let set N to 3

If $N = 3$, then array A has $3 \times 3 \times 3$, i.e., 27 elements
Therefore the nested loop iterates 27 times

			Iteration #
0	0	0	-> 1
1	0	0	-> 2
2	0	0	-> 3
0	0	1	-> 4
1	0	1	-> 5
2	0	1	-> 6
0	0	2	-> 7
1	0	2	-> 8
2	0	2	-> 9

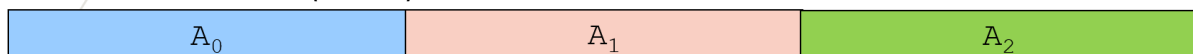
			Iteration #
0	1	0	-> 10
1	1	0	-> 11
2	1	0	-> 12
0	1	1	-> 13
1	1	1	-> 14
2	1	1	-> 15
0	1	2	-> 16
1	1	2	-> 17
2	1	2	-> 18

			Iteration #
0	2	0	-> 19
1	2	0	-> 20
2	2	0	-> 21
0	2	1	-> 22
1	2	1	-> 23
2	2	1	-> 24
0	2	2	-> 25
1	2	2	-> 26
2	2	2	-> 27

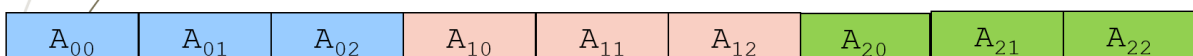
Solution (cont'd)

► Let's now draw our 3D array A in memory?

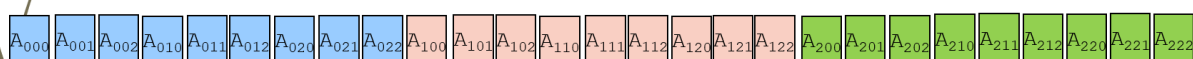
► Start with first (outer) dimension -> row: $A[N]$



► Then attend to the next dimension -> $A[N][N]$



► Then attend to the innermost dimension -> $A[N][N][N]$



```
int sumArray(int A[N][N][N])
{
    int i, j, k, sum = 0;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[k][i][j];
    return sum;
}
```

If $N = 3$

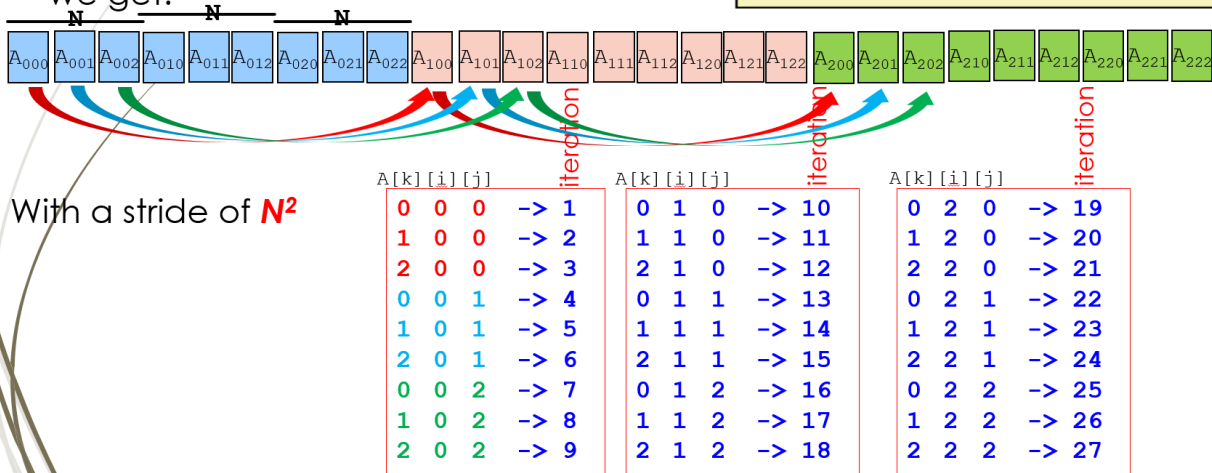
Solution (cont'd)

- Determining the memory access pattern we get:

```
int sumArray(int A[N][N][N]) If N = 3
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[k][i][j];

    return sum;
}
```



NOTE: Using $N = 4$ would better (more clearly) illustrate the stride of N^2 .

2. Rearrange its loops such that the function scans the 3D array **A** with a **stride of 1** memory access pattern and thus has good spatial locality.

Solution (cont'd)

2. Can we rearrange the loops such that the function scans the 3D array **A** with a **stride of 1** access pattern (and thus has **good spatial locality**)?

► A **stride of 1** would look like this (again, if $N = 3$):



Rearranging either the loop headers or the body of the loops, we get a few possible solutions:

Solution (cont'd)

A few possible solutions:

Possible solution 1:

```
int sumArray(int A[N][N][N])
{
    int i, j, k, sum = 0;

    for (k = 0; k < N; k++)
        for (i = 0; i < N; i++)
            for (j = 0; j < N; j++)
                sum += A[k][i][j];
    return sum;
}
```

Initial function:

```
int sumArray(int A[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[k][i][j];
    return sum;
}
```

Possible solution 2:

```
int sumArray(int A[N][N][N])
{
    int i, j, k, sum = 0;

    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            for (k = 0; k < N; k++)
                sum += A[i][j][k];
    return sum;
}
```

Q4 (4 points)

Microbenchmark data sets and graph from our Lab 12 data

In Lab 12, you are asked to microbenchmark code and to graph your results.

In this question, upload the completed data sheet you built in Lab 12 as well as the graph you produced.

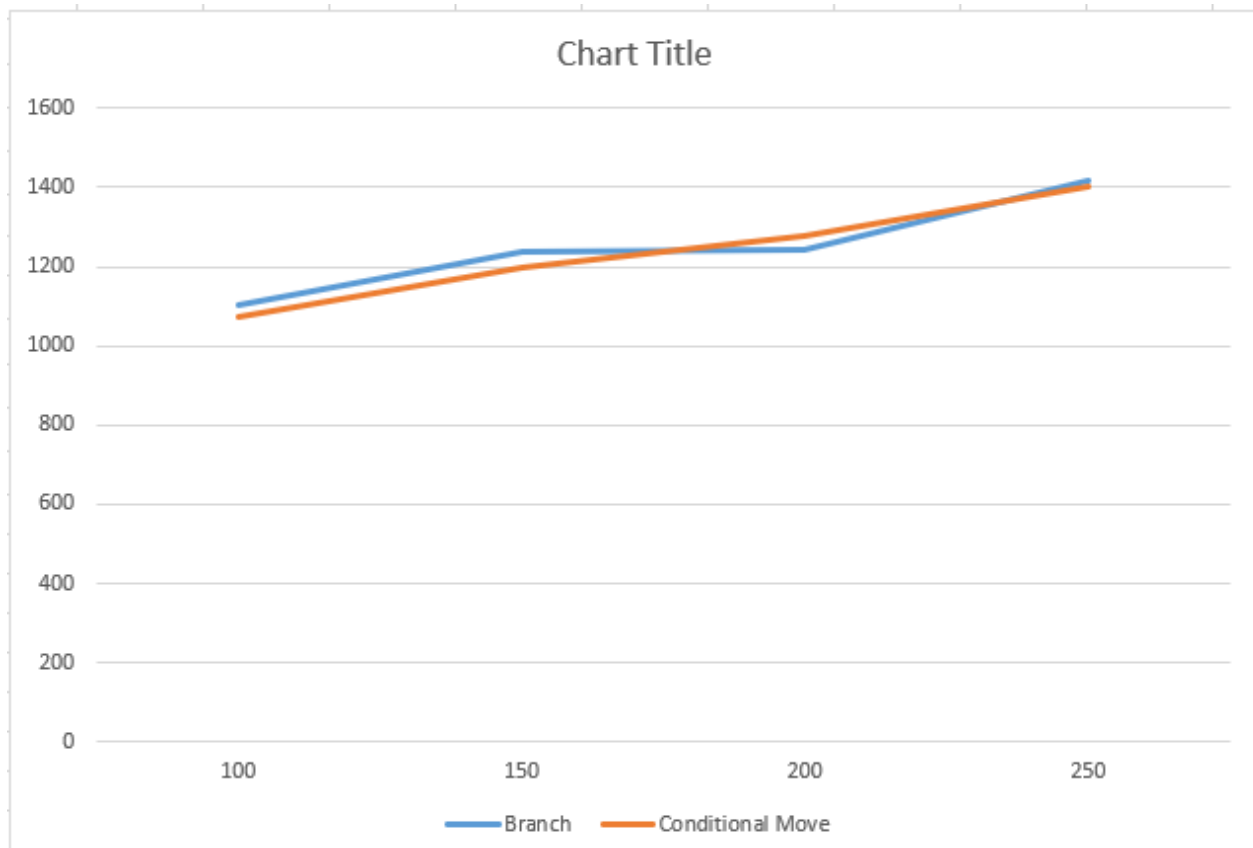
Make sure you graph all data sets onto the same graph and label each resulting line. This will ease comparison and allow you to promptly reach a conclusion.

Looking at your graph, what do you conclude?

Solution:

Branch						
N	t1	t2	t3	t4	t5	μ
100	1111	1080	1128	1084	1118	1104.2
150	1308	1212	1263	1185	1212	1236
200	1239	1258	1239	1237	1244	1243.4
250	1473	1383	1384	1416	1419	1415
Conditional Move						
N	t1	t2	t3	t4	t5	μ
100	1097	1068	1062	1084	1065	1075.2
150	1244	1317	1148	1131	1146	1197.2
200	1350	1265	1260	1261	1259	1279
250	1375	1310	1358	1641	1328	1402.4

	Branch	Conditional Move
100	1104.2	1075.2
150	1236	1197.2
200	1243.4	1279
250	1415	1402.4



Branch CPE	1407.639
Cmov CPE	1395.232

Conclusion:

When using a branch instruction (blue line on the graph), the CPE, i.e., the slope, varies. There may be many factors causing this variation, but we can ascertain that one of these factors is related to the instruction itself since this is the only difference between the two programs being tested. With a branch instruction, mispredictions can occur requiring the microprocessor to “waste” clock cycles, therefore needing more clock cycles to process some of these elements, hence increasing the CPE, i.e., the slope. Also, as the program processes more elements, the chance for the microprocessor to mispredict increases. This may explain the higher CPE of the last section of the blue line on the graph.

When using a conditional move instruction (orange line on the graph), the CPE, i.e., the slope, is constant for all values of N, i.e., the number of clock cycles required to process each element is constant.

Conclusion: The conditional move instruction does not impact the CPE, as the branch instruction does.

I am unsure as to why there is a flat section to the blue line on the graph where the slope is 0. This would require microbenchmarking again. 😞